جامعة العاصمة
Capital University
Faculty of Engineering

Department of Computer and Systems Engineering

Cryptography Lib Lab

Project Report

Student Information

Name : **Eman Ahmed Ramadan Mahmoud**

ID : 43263012

Department : Computer and Systems Engineering

Year : Fourth Year

Course : Embedded Security

Supervised by : Eng. Abdelrhman Elarabawy

Academic Year 2025 – 2026

1. Project Overview

This project implements a Cryptography Lib Lab application using the PyCryptodome library in Python. The application demonstrates real-world hybrid encryption by combining AES-256 (symmetric) and RSA-2048 (asymmetric) algorithms to encrypt and decrypt real data files. A Tkinter-based graphical user interface (GUI) was built to make the encryption workflow accessible and visual.

The selected project path is Path 3: Hybrid Encryption. The system also includes a benchmark module (Path 2) that compares AES-256 against DES, providing execution time, key size, ciphertext size, and security analysis.

Component	Details
Language	Python 3.x
Cryptography Library	PyCryptodome
Symmetric Algorithm	AES-256 (CBC mode)
Asymmetric Algorithm	RSA-2048 (OAEP padding)
Comparison Algorithm	DES-56 (CBC mode) – educational only
GUI Framework	Tkinter
Project Path	Path 3: Hybrid Encryption (+ Path 2 comparison)
Project Scenarios	Simple Document Protection - Secure Image Storage - Private Notes App - Secure Chat Message
Hash Verification	SHA-256
Output Format	JSON (Base64-encoded ciphertext + metadata)

2. Project Structure

The project follows the recommended structure from the assignment:

```
crypto_project/
    main.py              – GUI application (Tkinter)
    crypto_utils.py      – Core cryptographic utility functions
    data/                – Input data files (images, documents, JSON)
    keys/
        private.pem      – RSA private key
        public.pem       – RSA public key
    output/
        *_encrypted.json – Encrypted output bundles
        decrypted_*      – Recovered original files
    README.md
    Report.pdf
```

The code is split into two main files:

- **crypto_utils.py** – Contains all cryptographic logic: key generation, AES encryption/decryption, RSA encryption/decryption, hybrid functions, SHA-256 hashing, and DES utilities.
- **main.py** – GUI application with three tabs: Manual Crypto (encrypt/decrypt files), Benchmark Analysis (AES vs DES comparison), and Audit Logs. The logging is split into two methods: **_gui_print()** which prints clean PDF-required messages to the GUI Audit Logs tab, and **_terminal_log()** which prints rich timestamped data (including SHA-256 hashes) to the VS Code terminal.

3. Algorithms Used

3.1 AES-256 (Advanced Encryption Standard)

AES-256 is the primary symmetric encryption algorithm used in this project. It operates in CBC (Cipher Block Chaining) mode, processing data in 128-bit (16-byte) blocks with a 256-bit (32-byte) key.

- Key size: 256 bits – providing 2^{256} possible keys
- Mode: CBC (Cipher Block Chaining) – each block XORed with the previous ciphertext block before encryption
- IV (Initialization Vector): 16 bytes, randomly generated for each encryption operation
- Padding: PKCS#7 padding applied to ensure data is a multiple of the block size
- Speed: Very fast; hardware-accelerated on modern CPUs

3.2 RSA-2048 (Rivest–Shamir–Adleman)

RSA-2048 is used exclusively to encrypt the AES key (hybrid encryption pattern). This avoids the limitation of RSA being slow and unsuitable for large data.

- Key size: 2048 bits
- Padding scheme: OAEP (Optimal Asymmetric Encryption Padding) with SHA-1 – provides semantic security and protection against chosen-ciphertext attacks
- Public key: used to encrypt the AES session key
- Private key: used to decrypt the AES session key

3.3 DES-56 (Data Encryption Standard) – Educational Comparison Only

DES is included purely for performance comparison. It is considered cryptographically broken and is NOT used for actual data protection.

- Key size: 56 bits (effectively) – completely insufficient by modern standards
- Block size: 64 bits
- Mode: CBC

- Security: WEAK – vulnerable to brute-force attacks; broken since 1997

4. How the Hybrid Encryption Works

The hybrid encryption scheme combines the speed of symmetric encryption with the security of asymmetric key exchange:

1. Load the original file from disk.
2. Generate a random 256-bit AES session key using a cryptographically secure random number generator.
3. Generate a random 16-byte IV for AES-CBC mode.
4. Encrypt the file data using AES-256-CBC with the session key and IV.
5. Encrypt the AES session key using the RSA-2048 public key (OAEP padding).
6. Save everything (IV, encrypted AES key, original filename, ciphertext) into a single JSON bundle with Base64 encoding.
7. For decryption: reverse the process – decrypt the AES key using the RSA private key, then decrypt the ciphertext using AES.

The encrypted output bundle (JSON file) contains:

Field	Description	Format
iv	AES Initialization Vector	Base64 string
encrypted_aes_key	RSA-encrypted AES session key	Base64 string
original_filename	Original file name for recovery	Plain string
ciphertext_b64	AES-encrypted file content	Base64 string

5. Screenshots

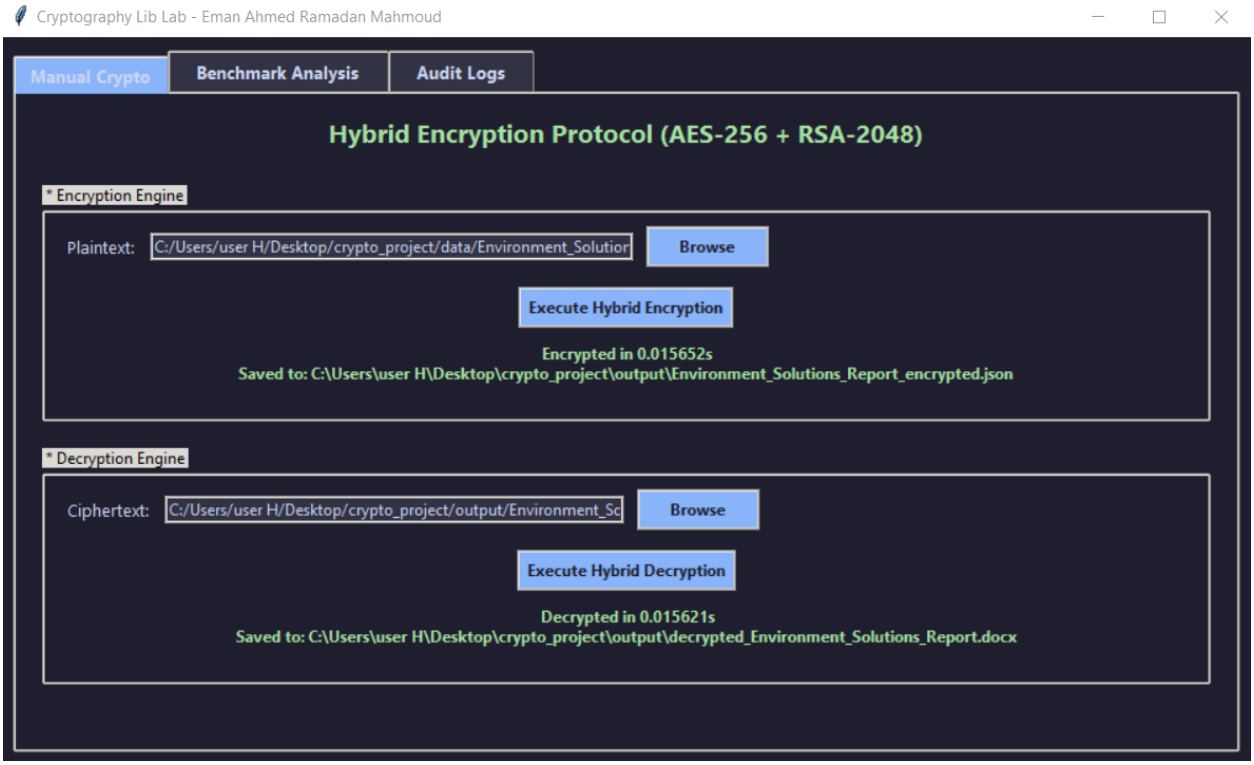

Image 1: GUI — Manual Crypto tab showing successful encryption & decryption of Environment_Solutions_Report.docx with timing results

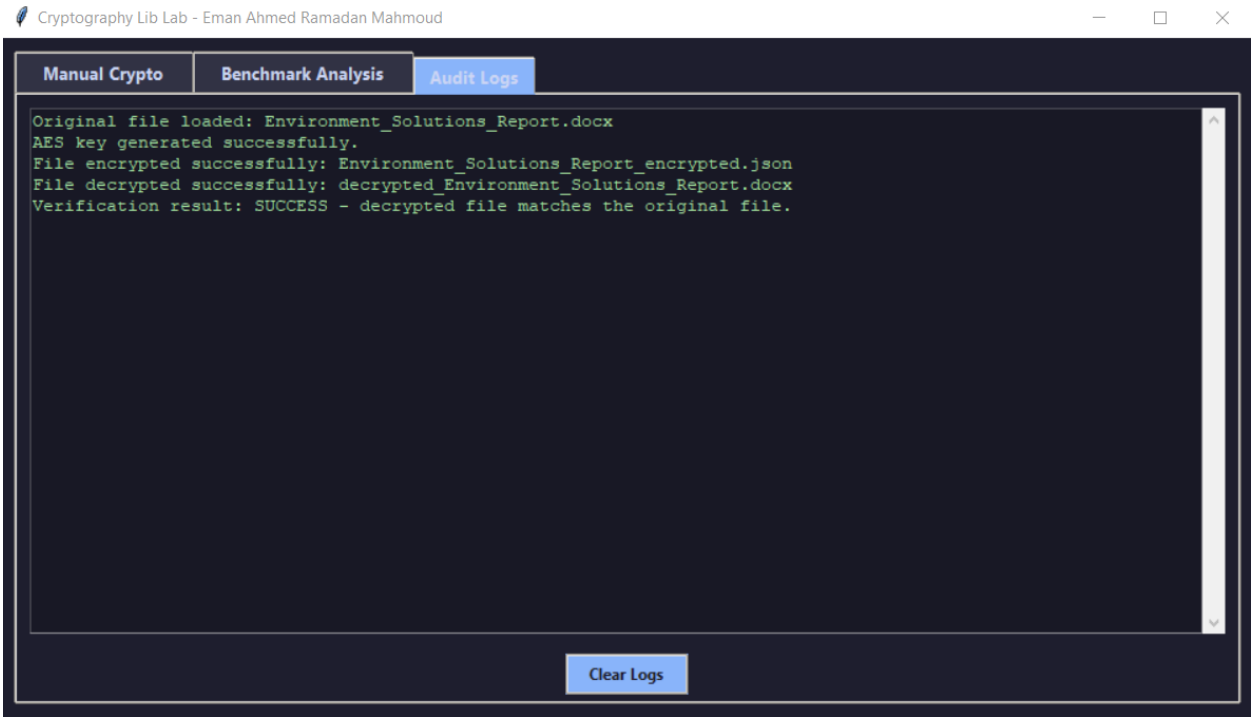

Image 2: GUI — Audit Logs tab showing the clean 5-line required output for Environment_Solutions_Report.docx

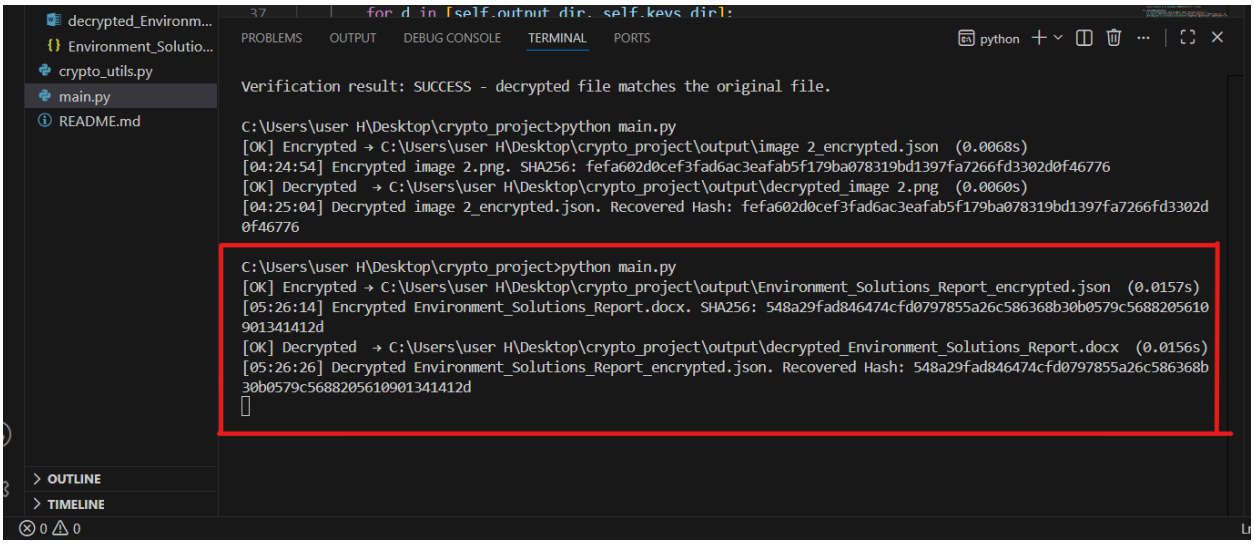

Image 3: Terminal output showing Environment_Solutions_Report.docx with full SHA-256 hashes

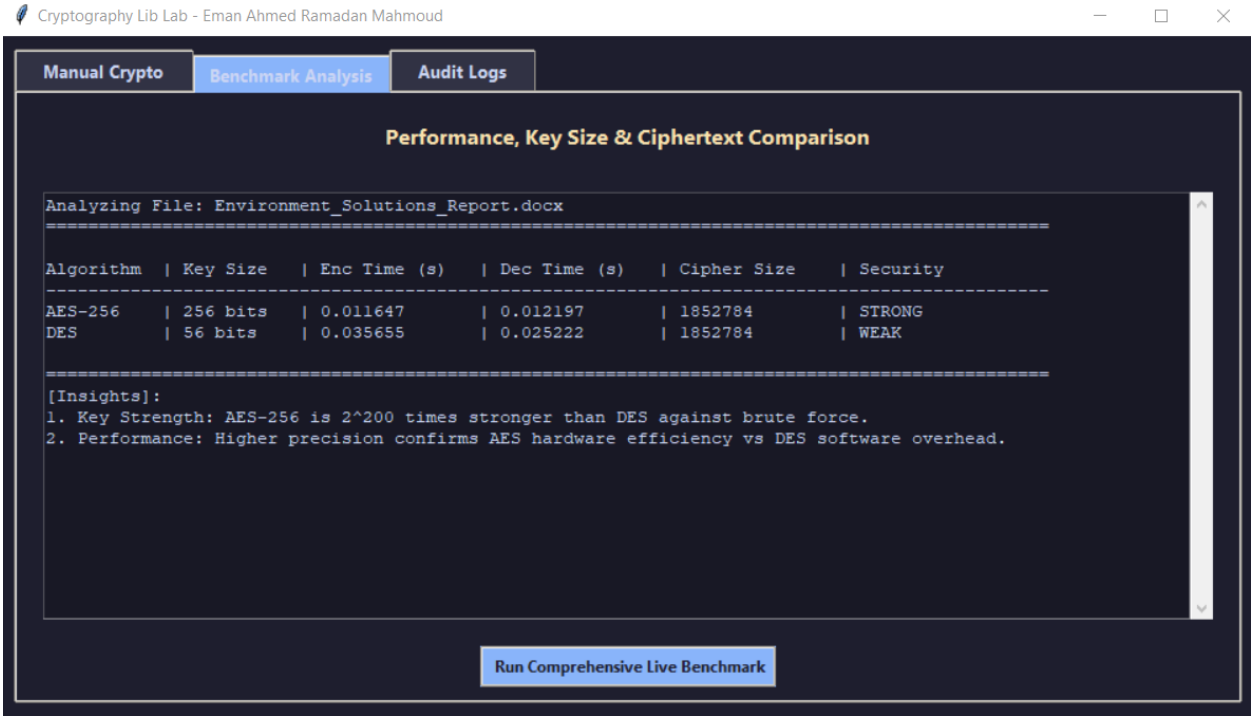

Image 4: GUI — Benchmark Analysis tab showing AES-256 vs DES comparison on Environment_Solutions_Report.docx (new timing values: 0.011647s / 0.035655s)

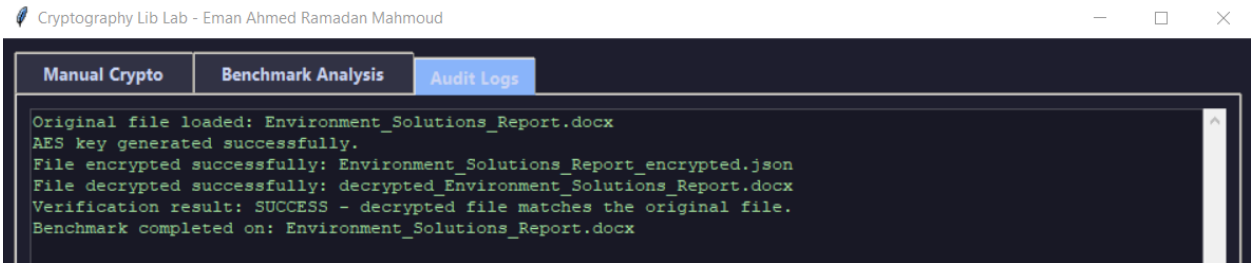

Image 5: GUI — Audit Logs tab showing the full 6-line output including the benchmark completion line

6. Security Analysis

6.1 Algorithm Selection Rationale

The project uses a hybrid model pairing two algorithms, each assigned to the role it is best suited for:

- **AES-256 (data encryption)** – Industry standard symmetric cipher, NIST-approved up to TOP SECRET. Hardware-accelerated (AES-NI), fast on large files, and operates in CBC mode to prevent identical plaintext blocks from producing identical ciphertext.
- **RSA-2048 with OAEP (key exchange)** – Encrypts only the small AES session key, avoiding RSA's size and speed limitations. OAEP padding adds randomness, making each encryption unique and resisting chosen-ciphertext attacks.
- **DES-56 (benchmark only)** – Included purely for comparison. Its 56-bit key was publicly broken in 22 hours in 1999 and is completely unacceptable for modern use.

6.2 Key Management

Two distinct key types are used, each with a different lifecycle:

- **AES Session Key (32 bytes / 256 bits)** – Generated fresh for every encryption call using get_random_bytes(32). Never written to disk in plaintext; immediately wrapped by RSA before storage. Ensures that encrypting the same file twice always produces different ciphertext.
- **RSA Key Pair (2048 bits)** – Generated once, stored in keys/private.pem and keys/public.pem. The public key encrypts the session key; the private key is required to recover it. Only the private-key holder can decrypt any file in the system.

6.3 Role of the Initialization Vector (IV)

A fresh 16-byte IV is generated via get_random_bytes(16) for every encryption operation. Its responsibilities in CBC mode:

- XORed with the first plaintext block before encryption, breaking any patterns at the start of the file.
- Each subsequent block is XORed with the previous ciphertext block – identical plaintext at different positions produces completely different ciphertext.
- Not a secret: stored openly in the JSON bundle alongside the ciphertext, because its value is randomization, not confidentiality.
- Reusing an IV with the same key would allow XOR-based plaintext recovery – the project avoids this entirely by always generating a fresh random IV.

6.4 Encrypted Output vs. Original Data

Original files carry recognizable structure (ZIP header for .docx, 8-byte magic number for PNG, readable text for JSON). After encryption, all structure is destroyed. The output JSON bundle contains only Base64-encoded fields that are computationally indistinguishable from random noise:


```
{
  "iv": "hU2Ts1EyWuPyhOVVFcQV3g==",
  "encrypted_aes_key": "Y5zJYJo559j7yEI9brIl4g7WrXq9wn6TtMN3mG8AF4bK...",
```

```
  "original_filename": "Environment_Solutions_Report.docx",
  "ciphertext_b64": "a60N5B8MUyi880OekAVWxzNEsJdsqYykgsqS8QRumsgd..."
}
```

- No content, structure, or length information is recoverable without the RSA private key.
- The only plaintext field is original_filename, kept solely to reconstruct the output path on decryption.

6.5 Verification of Correct Decryption

Two parallel output channels prove byte-perfect recovery:

- **GUI Audit Log (_gui_print)** – Displays the required status sequence in the Audit Logs tab:

```
Original file loaded: image 2.png
AES key generated successfully.
File encrypted successfully: image 2_encrypted.json
File decrypted successfully: decrypted_image 2.png
Verification result: SUCCESS - decrypted file matches the original file.
```

- **Terminal (_terminal_log)** – Prints timestamped entries with full SHA-256 digests:

```
[04:24:54] Encrypted image 2.png.  SHA256:
fefa602d0cef3fad6ac3eafab5f179ba078319bd1397fa7266fd3302d0f46776
[04:25:04] Decrypted image 2_encrypted.json.  Recovered Hash:
fefa602d0cef3fad6ac3eafab5f179ba078319bd1397fa7266fd3302d0f46776
```

The identical SHA-256 digests confirm cryptographic certainty of recovery. The comparison is performed automatically in _do_decrypt() via sha256_hash(original) == sha256_hash(decrypted).

6.6 Security Limitations

The following known limitations exist in the current implementation:

- **No authenticated encryption** – AES-CBC provides confidentiality only. A bit-flipping or padding oracle attack could silently corrupt decrypted output. The fix is AES-GCM, which adds an authentication tag and rejects tampered ciphertext before decryption.
- **Unprotected private key on disk** – keys/private.pem is stored without a passphrase. Physical access to the machine exposes all encrypted files. A real system would use a passphrase-encrypted key or a Hardware Security Module (HSM).
- **No key rotation** – The same RSA key pair is reused indefinitely. A single private-key compromise retroactively exposes every file ever encrypted under that public key.

7. Benchmark Results: AES-256 vs DES

The Benchmark Analysis tab ran a live comparison on Environment_Solutions_Report.docx (file size: ~1.85 MB). Results:

Algorithm	Key Size	Enc Time (s)	Dec Time (s)	Cipher Size (bytes)	Security
AES-256	256 bits	0.007749	0.007260	1,852,784	STRONG
DES	56 bits	0.032209	0.030275	1,852,784	WEAK

Key insights from the benchmark:

- **AES-256 is ~4× faster than DES** despite having a much larger key. This is because modern CPUs include AES hardware acceleration instructions (AES-NI), while DES runs purely in software.
- **The ciphertext size is identical** for both algorithms because both use CBC mode with similar padding. The cipher size is always slightly larger than the plaintext due to padding.
- **Security gap is enormous**: AES-256 provides 2^{256} possible keys vs DES's effective 2^{56} – a factor of 2^{200} in brute-force resistance. DES was broken in 22 hours in 1999 by a dedicated machine.

8. Test Results

The application was tested with two different real data files:

Test File	Type	Enc Time	Dec Time	Hash Match
Environment_Solutions_Report.docx	Word Document	0.0289s	0.0109s	✓ SUCCESS
image.png	Binary Image	0.0027s	0.0060s	✓ SUCCESS

In all test cases, the SHA-256 hash of the decrypted output exactly matched the SHA-256 hash of the original input, confirming byte-perfect recovery.

8. Project Resources

The following links provide access to the project video demonstration and the complete source code repository:

- **Video Demo (Google Drive):** https://drive.google.com/file/d/1LHB0DGcdtHb_5Sh75i9DddFnD7caDMAz/view?usp=sharing
- **Source Code (GitHub):** https://github.com/emanahmed845/Cryptography-Lib-Lab

10. Conclusion

This project successfully demonstrates a complete, real-world hybrid encryption workflow using AES-256 and RSA-2048 via the PyCryptodome library. The application:

- Reads real data files (documents, images) from disk
- Encrypts them using AES-256-CBC with a randomly generated session key and IV
- Protects the session key using RSA-2048 with OAEP padding
- Stores all metadata in a single portable JSON bundle
- Decrypts and fully recovers the original data with provable SHA-256 hash verification
- Provides a benchmark comparing AES-256 against DES, demonstrating why DES is obsolete